

Parallel Acceleration and Performance Analysis of Monte Carlo Simulation for Gold Investment Risk Assessment Using OpenMP and MPI

Andhika Narawangsa Susilo
Institut Teknologi Bandung
Bandung, Indonesia
13222036@mahasiswa.itb.ac.id

Jovan Hosea H. N.
Institut Teknologi Bandung
Bandung, Indonesia
13622005@mahasiswa.itb.ac.id

Kayla Pramudio B. A.
Institut Teknologi Bandung
Bandung, Indonesia
13222117@mahasiswa.itb.ac.id

Abstract – Monte Carlo simulation is widely used for financial risk analysis because it can represent uncertainty through a large number of stochastic price paths. However, the computational cost becomes significant when each path consists of multiple time steps. This study compares OpenMP and MPI implementations of a Monte Carlo simulation for gold investment risk analysis using a 252-step Geometric Brownian Motion model. The system is implemented in C++ and evaluated using 1,000,000 simulation paths calibrated from historical gold-price data. The experiments were conducted on an Intel Core i5-12500H processor with 12 physical cores and 16 logical processors. OpenMP implements shared-memory parallelism using multiple threads, whereas MPI implements distributed-memory parallelism using multiple ranks. The evaluated worker configurations are 1, 2, 4, 8, 12, and 16. The fastest OpenMP configuration achieved an execution time of 3.179 s using 16 threads, corresponding to an 8.88× speedup relative to the one-thread baseline. The fastest MPI configuration achieved an execution time of 3.297 s using 16 ranks, corresponding to an 8.60× speedup relative to the one-rank baseline. Both implementations produced terminal-price estimates close to the analytical GBM expectation while also estimating risk metrics including probability of loss, Value at Risk, and Conditional Value at Risk.

Keywords—High-Performance Computing, OpenMP, MPI, Monte Carlo, Geometric Brownian Motion, Value at Risk.

I. INTRODUCTION

The primary challenge in financial quantitative computing, particularly in gold investment, lies in the inability of simple deterministic analytical formulas to comprehensively map risk distributions for real world assets characterized by stochastic dynamics and continuous market uncertainty. Calculating expected final prices alongside extreme tail-risk indicators, such as Value at Risk (VaR) for a one year horizon, cannot be sufficiently characterized by a single deterministic estimate alone.

This operational reality forces analysts to generate millions to billions of potential future market paths via Monte Carlo path generation. The computational load escalates dramatically when attempting high resolution asset pricing across a massive number of simulation paths. Processing it at a multimillion or billion iteration scale transforms the problem into a critical performance bottleneck that demands a high performance computing infrastructure to deliver rapid and actionable risk insights. The workload consists of 1,000,000 simulation paths, each discretized into 252 trading-day steps.

This study compares two parallel programming models for Monte Carlo-based gold investment risk analysis: OpenMP and MPI. OpenMP distributes independent simulation paths among multiple threads within a shared-memory environment, whereas MPI distributes simulation paths among multiple ranks with separate memory spaces. The implementations are evaluated using 1,000,000 simulation paths and 252 time steps per path on a local multicore processor.

II. BASELINE AND EXISTING COMPUTATIONAL APPROACHES

The standard computational baseline currently deployed by financial analysts to address stochastic probability problems relies on executing sequential Monte Carlo simulation. High level interpreted languages such as Python (leveraging Pandas or NumPy frameworks), the R language, or scripting macros inside Microsoft Excel are heavily utilized.

This conventional approach threads simulation iterations sequentially within a single threaded execution loop on a single CPU core. While highly straightforward to implement programmatically, this method suffers from severe scalability limits and prohibitive execution times. When confronted with institutional-grade computational requirements, such as evaluating millions independent simulation trajectories, the sequential approach under interpreted runtimes frequently triggers memory exhaustion errors or demands hours of runtime, rendering it obsolete for real-time strategic risk management and dynamic decision-making.

III. PARALLEL COMPUTATIONAL DESIGN

A. Parallelization Strategy

The Monte Carlo simulation is parallelized by exploiting the independence of individual simulation paths. Each path starts from the same calibrated initial parameters but uses an independent random-number sequence to generate a different gold-price trajectory. Therefore, the total number of simulation paths can be partitioned among multiple computational workers without requiring communication during the path-generation stage.

For a total of N simulation paths and p workers, each worker is assigned approximately N/p paths. Every worker independently performs the 252-step Geometric Brownian Motion calculation for its assigned paths and accumulates local statistical results, including the terminal-price sum,

squared-price sum, loss count, drawdown, minimum price, maximum price, and terminal-price histogram.

After all workers finish their assigned paths, the local statistics are aggregated into global results. The final aggregated histogram is then used to estimate percentile values, Value at Risk, and Conditional Value at Risk. This approach minimizes communication during the computationally intensive simulation phase and performs synchronization only during the final aggregation stage.

B. Unified Parallel Computational Architecture

The proposed computational architecture consists of four main stages: parameter initialization, workload partitioning, parallel path simulation, and result aggregation. During parameter initialization, the system reads the historical gold-price and risk-free-rate datasets to determine the initial price, risk-free rate, and annualized volatility. These parameters are shared or distributed to all computational workers before the simulation begins.

In the workload partitioning stage, the total Monte Carlo paths are divided among the available workers. Each worker initializes an independent random-number generator and simulates its assigned price paths using the 252-step Geometric Brownian Motion model. To avoid race conditions and reduce synchronization overhead, each worker stores terminal-price frequencies in a local histogram and maintains local statistical accumulators.

After the parallel simulation stage is completed, the local histograms and statistical accumulators are combined into a global distribution. The global results are then used to calculate the mean terminal price, standard deviation, probability of loss, drawdown, percentile values, Value at Risk, and Conditional Value at Risk.

The same logical architecture is implemented using two parallel programming models. In OpenMP, workers are implemented as threads within a shared-memory environment. In MPI, workers are implemented as ranks with separate memory spaces, where the final aggregation is performed through MPI communication primitives.

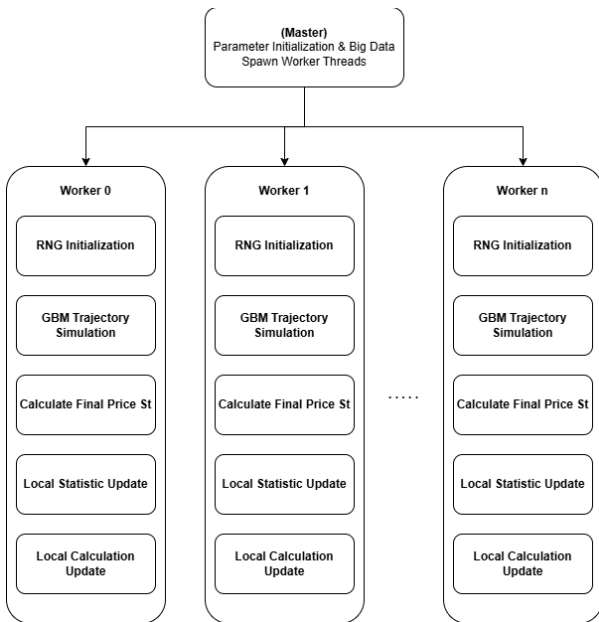


Fig 1. Parallel Worker Architecture

C. Computational Workflow

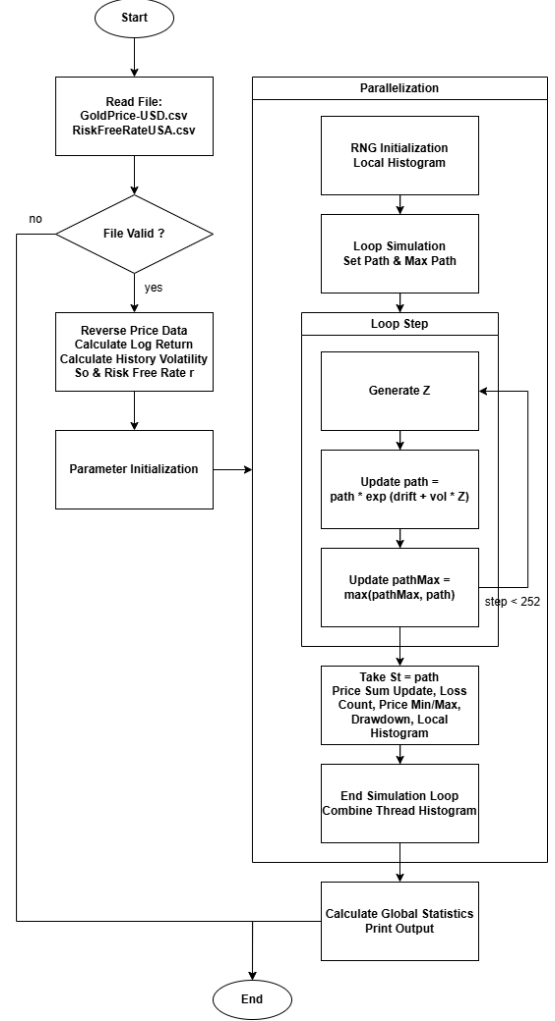


Fig 2. Monte Carlo Simulation Flowchart Diagram

The workflow is identical for both OpenMP and MPI during the Monte Carlo computation stage. The difference lies only in the execution mechanism: OpenMP assigns paths to threads through a shared-memory parallel loop, whereas MPI assigns paths to ranks and aggregates local results using message-passing operations.

- **Input Phase:** The system extracts raw historical parameters from the GoldPrice-USD.csv and RiskFreeRateUSA.csv datasets.
- **Pre-Computation Phase:** The orchestrator calculates the annualized historical volatility and the corresponding real-world risk-free rate.
- **Parallel Execution Phase:** Calibrated baseline variables trigger the parallel simulation loop. Each micro-thread autonomously evaluates the terminal exponential price step driven by independent standard normal random samples.
- **Aggregation Phase:** Frequency bin arrays are accumulated globally into the memory workspace of Rank 0 to reconstruct the terminal probability distribution.

IV. IMPLEMENTATION

The Monte Carlo risk-analysis system was implemented in ISO C++17 using two functionally equivalent parallel versions: OpenMP and MPI. The OpenMP implementation uses shared-memory parallelism, where simulation paths are distributed among multiple threads within a single process. The MPI implementation uses distributed-memory parallelism, where simulation paths are distributed among multiple ranks. Both implementations were compiled using optimization-enabled GCC-based toolchains with the -O3 optimization flag.

The input module reads historical gold-price and risk-free-rate data from semicolon-separated CSV files. Comma characters contained in numerical values are removed before the data are converted into floating-point values. The historical gold-price data are then used to calculate annualized volatility from daily logarithmic returns, while the latest gold price and risk-free rate are used as the initial GBM parameters.

Each Monte Carlo path represents one year of gold-price movement divided into 252 trading-day intervals. Therefore, the simulation applies the discretized Geometric Brownian Motion update iteratively for each time step:

$$S_T = S_0 \exp \left(\left(r - \frac{\sigma^2}{2} \right) \Delta t + \sigma \sqrt{\Delta t} Z \right)$$

where (S_t) is the gold price at time (t), (r) is the risk-free rate used as the drift parameter, (σ) is the annualized historical volatility, (Z_t) is a standard normal random variable, and ($\Delta t = 1/252$). Each simulation path is updated 252 times before its final terminal price is obtained.

Each computational worker initializes an independent 64-bit Mersenne Twister random-number generator, {std::mt19937_64}, with a worker-specific seed. In the OpenMP implementation, each thread maintains its own random-number generator and local histogram. In the MPI implementation, each rank maintains its own random-number generator and local histogram. This design avoids race conditions during random-number generation and histogram updates.

To reduce memory usage, the implementation does not store all terminal prices individually. Instead, each worker maps its terminal-price results into a fixed 10,000-bin local histogram. After the simulation stage is completed, the local histograms and statistical accumulators are combined into global results. In OpenMP, this aggregation is performed after the parallel region finishes, whereas in MPI, the local results are combined using reduction operations. The global histogram is then used to estimate percentile values, Value at Risk, and Conditional Value at Risk. Because the histogram uses fixed bins, the resulting percentile-based risk metrics are treated as efficient approximations of the terminal-price distribution.

V. EXPERIMENTAL RESULTS AND ANALYSIS

1. Experimental Configuration

Table I. Experimental Configuration

Parameter	Configuration
Processor	Intel Core i5-12500H, 12th Generation

Physical cores	12 cores
Logical processors	16 logical processors
Base clock	2.50 GHz
Parallel models	OpenMP and MPI
Worker configurations	1, 2, 4, 8, 12, and 16
Simulation paths	1,000,000
Time steps per path	252
Historical gold-price records	2,189 observations
Initial price (S_0)	\$4,482.32
Risk-free rate (r)	4.47%
Historical volatility	17.63%
Programming language	C++
OpenMP compilation	g++ -O3 -std=c++17 -fopenmp
MPI compilation	mpicxx -O3 -std=c++17
Operating system	Windows 11 Home Single Language 25H2
Memory	16GB

2. Computational Performance Comparison Analysis

The performance benchmark was conducted using 1,000,000 Monte Carlo simulation paths, with each path simulated over 252 Geometric Brownian Motion time steps. Both OpenMP and MPI implementations were evaluated using 1, 2, 4, 8, 12, and 16 workers under the same hardware and input-data conditions. The execution time, speedup, and parallel efficiency for each configuration are summarized in Table II.

The speedup and parallel efficiency calculated for each parallel framework is calculated using:

$$S_p = \frac{T_s}{T_p} \text{ and } E_p = \frac{S_p}{p} \times 100\%$$

With:

$$\begin{aligned} S_p &: \text{Speedup using } p \text{ workers} \\ T_s &: \text{Sequential execution time} \\ T_p &: \text{Execution Time with } p \text{ workers} \\ E_p &: \text{Efficiency of } p \text{ workers} \\ p &: \text{workers used} \end{aligned}$$

Table II. Experimental Results

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	28.22	28.36	1.00	1.00	100.00%	100.00%
2	13.45	13.43	2.10	2.11	104.87%	105.60%
4	6.78	7.40	4.16	3.84	104.01%	95.88%
8	4.88	5.44	5.78	5.21	72.30%	65.14%
12	3.68	3.52	7.66	8.05	63.86%	67.10%
16	3.18	3.30	8.88	8.60	55.47%	53.76%

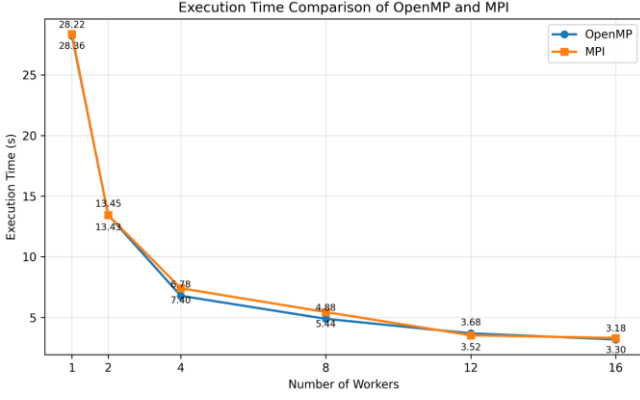


Fig 3. Execution Time vs Number of Workers

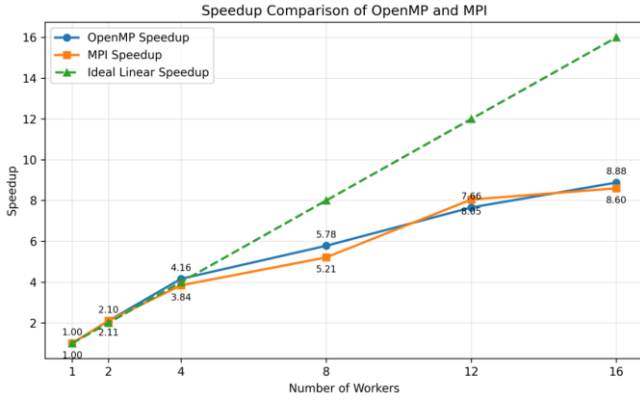


Fig 4. Speedup vs Number of Workers

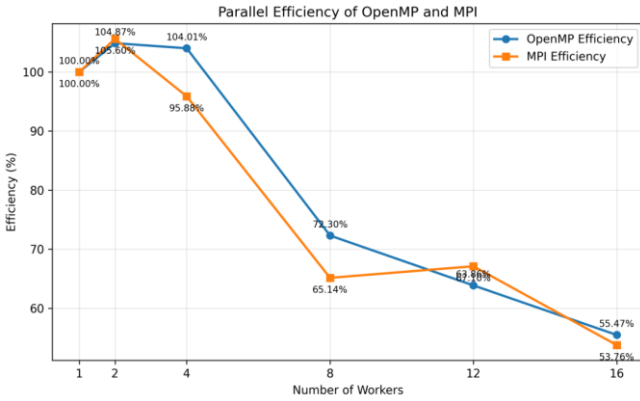


Fig 5. Parallel Efficiency vs Number of Workers

The OpenMP implementation reduced the execution time from 28.22 s using one thread to 3.18 s using 16 threads, corresponding to a maximum speedup of 8.88 $\times$. Similarly, the MPI implementation reduced the execution time from 28.36 s using one rank to 3.30 s using 16 ranks, achieving a maximum speedup of 8.60 $\times$. Both implementations demonstrate that Monte Carlo simulation is highly suitable for parallel execution because each simulation path can be calculated independently before the final aggregation stage.

As shown in Fig. 4 and Fig. 5, both OpenMP and MPI exhibit near-linear scaling at low worker counts. OpenMP achieved speedups of 2.10 $\times$ and 4.16 $\times$ using two and four threads, respectively, while MPI achieved speedups of 2.11 $\times$ and 3.84 $\times$ using two and four ranks. The slightly above-ideal efficiency values at low worker counts are likely caused by cache effects, CPU turbo boost behavior, and runtime

measurement variation rather than a true superlinear parallel performance improvement.

The performance gain becomes less linear at higher worker counts. At 16 workers, OpenMP achieved the lowest execution time of 3.18 s, while MPI required 3.30 s. The parallel efficiency decreased to 55.47% for OpenMP and 53.76% for MPI at 16 workers, as illustrated in Fig. 6. This reduction is caused by synchronization overhead, memory contention, operating-system scheduling, and increased competition for processor resources. In addition, the evaluated processor provides 12 physical cores and 16 logical processors; therefore, the additional workers beyond 12 primarily utilize logical processors rather than fully independent physical cores.

Overall, both OpenMP and MPI significantly accelerated the Monte Carlo simulation workload. OpenMP produced the best execution time on the evaluated local multicore system, while MPI showed comparable performance and provides a distributed-memory model that can be extended to multi-node computing environments. These results confirm that parallel programming is effective for accelerating large-scale stochastic simulations with a high number of independent computation paths.

3. Numerical Verification and Risk Metrics

a) Numerical Verification

The numerical validity of the simulation was evaluated by comparing the simulated mean terminal price with the analytical expectation of the Geometric Brownian Motion model:

$$E[S_T] = S_0 e^{(rT)}$$

Using the calibrated parameters, the analytical expected terminal price was \$4687.23. The fastest OpenMP configuration, using 16 threads, produced a simulated mean terminal price of \$4687.61, corresponding to a relative error of approximately 0.008%. The fastest MPI configuration, using 16 ranks, produced a simulated mean terminal price of \$4685.39, corresponding to a relative error of approximately 0.039%.

These small deviations indicate that both parallel implementations preserve the expected statistical behavior of the GBM model. The remaining difference from the analytical value is expected because Monte Carlo simulation uses finite random samples and different worker configurations generate independent random-number streams.

b) Risk Metric Estimation

The terminal-price distribution was used to estimate the probability of loss, Value at Risk, and Conditional Value at Risk. The probability of loss is defined as the proportion of simulation paths with a terminal price below the initial gold price (S_0). VaR represents the estimated loss threshold at a specified confidence level, while CVaR represents the average loss within the worst tail region beyond the VaR threshold.

Table III summarizes the numerical verification and risk metrics obtained from the fastest OpenMP and MPI configurations. Both implementations produced closely

comparable risk estimates, with small differences caused by stochastic sampling variation and the use of independent random-number sequences.

Table III. Numerical Verification and Risk Metrics of the Parallel Configurations

Metric	OpenMP (16 Threads)	MPI (16 Ranks)
Execution Time (s)	3.18	3.297
Simulated Mean Price	\$4,687.61	\$4,685.39
Analytical Mean Price	\$4,687.23	\$4,687.23
Relative Error	0.01%	0.04%
Final Price Std Dev	\$832.44	\$832.68
Probability of Loss	43.24%	43.52%
Average Drawdown	10.91%	10.93%
VaR 95%	\$1,029.08	\$1,030.57
VaR 99%	\$1,419.46	\$1,422.44
CVaR 95%	\$1,267.71	\$1,269.74

The results indicate that the estimated probability of loss is approximately 43%, meaning that a substantial portion of the simulated one-year scenarios end below the initial gold price. At the 95% confidence level, the estimated VaR is approximately \$1030 per unit of gold-price exposure, while the CVaR indicates that the average loss within the worst 5% of scenarios is approximately \$1268–\$1270. Because the percentile values are derived from a fixed-bin histogram, the reported VaR and CVaR values should be interpreted as efficient approximations of the simulated terminal-price distribution.

The source code for the simulation is available [here](#)

VI. CONCLUSION

This study compared OpenMP and MPI implementations for a Monte Carlo gold investment simulation with 1,000,000 paths and 252 GBM time steps per path. OpenMP achieved the best execution time of 3.18 s using 16 threads, equivalent to an 8.88 $\times$ speedup, while MPI achieved 3.30 s using 16 ranks with an 8.60 $\times$ speedup.

Both implementations reduced runtime by more than 88% relative to their single-worker baselines and produced mean terminal prices with relative errors below 0.04% from the analytical GBM expectation. The resulting terminal-price distributions were also used to estimate loss probability, VaR, and CVaR.

OpenMP provided the lowest runtime on the tested multicore laptop, whereas MPI produced comparable results and remains suitable for extension to distributed multi-node systems.

REFERENCES

- [1] N. A. Zakaria, M. F. Ahmad, and M. S. M. Kasihmuddin, "Modeling and forecasting gold price using Geometric Brownian Motion," AIP Conf. Proc., vol. 2138, no. 1, pp. 030018-1–030018-6, Aug. 2019.
- [2] S. Maruddani and A. Prahutama, "Value at Risk portfolio using Monte Carlo simulation," J. Phys.: Conf. Ser., vol. 855, no. 1, pp. 012027-1–012027-7, May 2017.
- [3] V. Alexandrov, "Parallel Monte Carlo algorithms for financial derivatives pricing," Math. Comput. Simul., vol. 47, no. 2, pp. 113–122, Aug. 1998.
- [4] M. B. Giles, "Multi-level Monte Carlo path simulation," Oper. Res., vol. 56, no. 3, pp. 607–617, Jun. 2008.
- [5] T. J. Linsmeier and N. D. Pearson, "Value at Risk," Financ. Anal. J., vol. 56, no. 2, pp. 47–67, Mar. 2000.

APPENDIX A OPENMP EXECUTION OUTPUTS

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ export OMP_NUM_THREADS=1

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ ./main_openmp.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo parallel OpenMP (252-step GBM)...
OpenMP Threads : 1

=== MONTE CARLO OPENMP RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 28.216000 s

Simulation Price Mean : $4686.393335
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $832.615970
Final Price Min : $1902.851855
Final Price Max : $10196.698333
Loss Probability : 43.482300%
Average Drawdown : 10.926519%

=== FINAL PRICE PERCENTILE ===
P1 : $3059.885000
P5 (worst 5%) : $3451.755000
P10 (worst 10%) : $3681.215000
P25 : $4096.925000
P50 (Median) : $4613.955000
P75 : $5196.545000
P90 : $5783.605000
P95 (best 5%) : $6165.045000

=== RISK METRICS ===
VaR 95% : $1030.565000
VaR 99% : $1422.435000
CVaR 95% approx. : $1269.763431

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$
```

Fig. A1. OpenMP execution output using 1 thread

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ export OMP_NUM_THREADS=2

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ ./main_openmp.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo parallel OpenMP (252-step GBM)...
OpenMP Threads : 2

=== MONTE CARLO OPENMP RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 13.453000 s

Simulation Price Mean : $4685.902338
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $831.075446
Final Price Min : $2026.364795
Final Price Max : $10466.685340
Loss Probability : 43.452900%
Average Drawdown : 10.922845%

=== FINAL PRICE PERCENTILE ===
P1 : $3061.375000
P5 (worst 5%) : $3454.735000
P10 (worst 10%) : $3681.215000
P25 : $4096.925000
P50 (Median) : $4613.955000
P75 : $5196.545000
P90 : $5782.115000
P95 (best 5%) : $6162.065000

=== RISK METRICS ===
VaR 95% : $1027.585000
VaR 99% : $1420.945000
CVaR 95% approx. : $1266.541217

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ |
```

Fig. A2. OpenMP execution output using 2 threads

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ export OMP_NUM_THREADS=4

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ ./main_openmp.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo parallel OpenMP (252-step GBM)...
OpenMP Threads : 4

=== MONTE CARLO OPENMP RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 6.782000 s

Simulation Price Mean : $4686.191266
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $832.451906
Final Price Min : $1893.035862
Final Price Max : $10723.303939
Loss Probability : 43.503600%
Average Drawdown : 10.918432%

=== FINAL PRICE PERCENTILE ===
P1 : $3061.375000
P5 (worst 5%) : $3453.245000
P10 (worst 10%) : $3681.215000
P25 : $4096.925000
P50 (Median) : $4613.955000
P75 : $5196.545000
P90 : $5785.095000
P95 (best 5%) : $6166.535000

=== RISK METRICS ===
VaR 95% : $1029.075000
VaR 99% : $1420.945000
CVaR 95% approx. : $1266.919141

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ |
```

Fig. A3. OpenMP execution output using 4 threads

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ export OMP_NUM_THREADS=8

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ ./main_openmp.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo parallel OpenMP (252-step GBM)...
OpenMP Threads : 8

=== MONTE CARLO OPENMP RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 4.876000 s

Simulation Price Mean : $4687.618775
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $833.699457
Final Price Min : $2040.109919
Final Price Max : $12172.411054
Loss Probability : 43.400200%
Average Drawdown : 10.920813%

=== FINAL PRICE PERCENTILE ===
P1 : $3061.375000
P5 (worst 5%) : $3451.755000
P10 (worst 10%) : $3681.215000
P25 : $4096.925000
P50 (Median) : $4613.445000
P75 : $5198.035000
P90 : $5785.095000
P95 (best 5%) : $6171.005000

=== RISK METRICS ===
VaR 95% : $1030.565000
VaR 99% : $1420.945000
CVaR 95% approx. : $1269.169786

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$
```

Fig. A4. OpenMP execution output using 8 threads


```

M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ export OMP_NUM_THREADS=12

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ ./main_openmp.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo parallel OpenMP (252-step GBM)...
OpenMP Threads : 12

=== MONTE CARLO OPENMP RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 3.682000 s

Simulation Price Mean : $4688.412921
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $833.022389
Final Price Min : $2003.040936
Final Price Max : $10439.001511
Loss Probability : 43.401600%
Average Drawdown : 10.907413%

=== FINAL PRICE PERCENTILE ===
P1 : $3064.355000
P5 (worst 5%) : $3454.735000
P10 (worst 10%) : $3682.705000
P25 : $4098.415000
P50 (Median) : $4615.445000
P75 : $5199.525000
P90 : $5786.585000
P95 (best 5%) : $6168.025000

=== RISK METRICS ===
VaR 95% : $1027.585000
VaR 99% : $1417.965000
CVaR 95% approx. : $1266.039266

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ |

```

Fig. A5. OpenMP execution output using 12 threads

```

M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ export OMP_NUM_THREADS=16

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ ./main_openmp.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo parallel OpenMP (252-step GBM)...
OpenMP Threads : 16

=== MONTE CARLO OPENMP RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 3.179000 s

Simulation Price Mean : $4687.614392
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $832.436244
Final Price Min : $1954.762251
Final Price Max : $10568.702486
Loss Probability : 43.323600%
Average Drawdown : 10.907587%

=== FINAL PRICE PERCENTILE ===
P1 : $3062.865000
P5 (worst 5%) : $3453.245000
P10 (worst 10%) : $3681.215000
P25 : $4096.925000
P50 (Median) : $4616.935000
P75 : $5198.035000
P90 : $5783.605000
P95 (best 5%) : $6166.535000

=== RISK METRICS ===
VaR 95% : $1029.075000
VaR 99% : $1419.455000
CVaR 95% approx. : $1267.709258

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ |

```

Fig. A6. OpenMP execution output using 16 threads

APPENDIX B

MPI EXECUTION OUTPUTS

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ mpiexec -np 1 ./main_mpi.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0)      : $4482.320000
Risk-Free Rate (r)    : 4.470000%
History Volatility    : 17.630550%

Start Execution 1000000 Monte Carlo MPI (252-step GBM)...
MPI ranks count      : 1

=== MONTE CARLO MPI RESULT ===
Simulation Count      : 1000000
Step per Path         : 252
Execution Time        : 28.359638 s

Simulation Price Mean : $4687.988601
GBM Analytics Mean    : $4687.225219
Final Price Std Dev   : $831.161998
Final Price Min       : $1906.256471
Final Price Max       : $10708.604746
Loss Probability      : 43.372300%
Average Drawdown      : 10.894688%

=== FINAL PRICE PERCENTILE ===
P1                    : $3067.335000
P5 (worst 5%)         : $3454.735000
P10 (worst 10%)       : $3684.195000
P25                   : $4099.905000
P50 (Median)          : $4615.445000
P75                   : $5196.545000
P90                   : $5785.095000
P95 (best 5%)         : $6168.025000

=== RISK METRICS ===
VaR 95%               : $1027.585000
VaR 99%               : $1414.985000
CVar 95% approx.      : $1264.33156

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$
```

Fig. B1. MPI execution output using 1 rank

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ mpiexec -np 2 ./main_mpi.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0)      : $4482.320000
Risk-Free Rate (r)    : 4.470000%
History Volatility    : 17.630550%

Start Execution 1000000 Monte Carlo MPI (252-step GBM)...
MPI ranks count      : 2

=== MONTE CARLO MPI RESULT ===
Simulation Count      : 1000000
Step per Path         : 252
Execution Time        : 13.427893 s

Simulation Price Mean : $4685.757170
GBM Analytics Mean    : $4687.225219
Final Price Std Dev   : $831.786939
Final Price Min       : $2059.369307
Final Price Max       : $10671.553297
Loss Probability      : 43.463600%
Average Drawdown      : 10.932247%

=== FINAL PRICE PERCENTILE ===
P1                    : $3064.355000
P5 (worst 5%)         : $3451.755000
P10 (worst 10%)       : $3681.215000
P25                   : $4095.435000
P50 (Median)          : $4613.955000
P75                   : $5196.545000
P90                   : $5782.115000
P95 (best 5%)         : $6166.535000

=== RISK METRICS ===
VaR 95%               : $1030.565000
VaR 99%               : $1417.965000
CVar 95% approx.      : $1268.081877

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ |
```

Fig. B2. MPI execution output using 2 ranks.

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ mpiexec -np 4 ./main_mpi.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0)      : $4482.320000
Risk-Free Rate (r)    : 4.470000%
History Volatility    : 17.630550%

Start Execution 1000000 Monte Carlo MPI (252-step GBM)...
MPI ranks count      : 4

=== MONTE CARLO MPI RESULT ===
Simulation Count      : 1000000
Step per Path         : 252
Execution Time        : 7.395211 s

Simulation Price Mean : $4687.519949
GBM Analytics Mean    : $4687.225219
Final Price Std Dev   : $832.312597
Final Price Min       : $1980.182479
Final Price Max       : $10777.796155
Loss Probability      : 43.372800%
Average Drawdown      : 10.908518%

=== FINAL PRICE PERCENTILE ===
P1                    : $3059.885000
P5 (worst 5%)         : $3453.245000
P10 (worst 10%)       : $3681.215000
P25                   : $4098.415000
P50 (Median)          : $4615.445000
P75                   : $5199.525000
P90                   : $5783.605000
P95 (best 5%)         : $6165.045000

=== RISK METRICS ===
VaR 95%               : $1029.075000
VaR 99%               : $1422.435000
CVar 95% approx.      : $1267.186417

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$
```

Fig. B3. MPI execution output using 4 ranks.

```
M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ mpiexec -np 8 ./main_mpi.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0)      : $4482.320000
Risk-Free Rate (r)    : 4.470000%
History Volatility    : 17.630550%

Start Execution 1000000 Monte Carlo MPI (252-step GBM)...
MPI ranks count      : 8

=== MONTE CARLO MPI RESULT ===
Simulation Count      : 1000000
Step per Path         : 252
Execution Time        : 5.442339 s

Simulation Price Mean : $4687.384665
GBM Analytics Mean    : $4687.225219
Final Price Std Dev   : $832.740169
Final Price Min       : $2044.787583
Final Price Max       : $11330.015998
Loss Probability      : 43.403100%
Average Drawdown      : 10.911966%

=== FINAL PRICE PERCENTILE ===
P1                    : $3062.865000
P5 (worst 5%)         : $3451.755000
P10 (worst 10%)       : $3681.215000
P25                   : $4098.415000
P50 (Median)          : $4615.445000
P75                   : $5198.035000
P90                   : $5783.605000
P95 (best 5%)         : $6166.535000

=== RISK METRICS ===
VaR 95%               : $1030.565000
VaR 99%               : $1419.455000
CVar 95% approx.      : $1268.352044

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$
```

Fig. B4. MPI execution output using 8 ranks.


```

M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ mpiexec -np 12 ./main_mpi.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo MPI (252-step GBM)...
MPI ranks count : 12

=== MONTE CARLO MPI RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 3.521939 s

Simulation Price Mean : $4686.379611
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $833.138016
Final Price Min : $1914.025825
Final Price Max : $10837.394942
Loss Probability : 43.514300%
Average Drawdown : 10.924464%

=== FINAL PRICE PERCENTILE ===
P1 : $3062.865000
P5 (worst 5%) : $3451.755000
P10 (worst 10%) : $3681.215000
P25 : $4096.925000
P50 (Median) : $4613.955000
P75 : $5196.545000
P90 : $5785.095000
P95 (best 5%) : $6166.535000

=== RISK METRICS ===
VaR 95% : $1030.565000
VaR 99% : $1419.455000
CVaR 95% approx. : $1268.127680

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ |

```

Fig. B5. MPI execution output using 12 ranks.

```

M /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$ mpiexec -np 16 ./main_mpi.exe 1000000
=== REAL WORLD HISTORY DATA CALIBRATION===
Gold Price Days Count: 2189 days
Start Price ($0) : $4482.320000
Risk-Free Rate (r) : 4.470000%
History Volatility : 17.630550%

Start Execution 1000000 Monte Carlo MPI (252-step GBM)...
MPI ranks count : 16

=== MONTE CARLO MPI RESULT ===
Simulation Count : 1000000
Step per Path : 252
Execution Time : 3.296586 s

Simulation Price Mean : $4685.390750
GBM Analytics Mean : $4687.225219
Final Price Std Dev : $832.684072
Final Price Min : $2044.677824
Final Price Max : $11047.703328
Loss Probability : 43.522000%
Average Drawdown : 10.932989%

=== FINAL PRICE PERCENTILE ===
P1 : $3059.885000
P5 (worst 5%) : $3451.755000
P10 (worst 10%) : $3681.215000
P25 : $4095.435000
P50 (Median) : $4613.955000
P75 : $5195.055000
P90 : $5783.605000
P95 (best 5%) : $6166.535000

=== RISK METRICS ===
VaR 95% : $1030.565000
VaR 99% : $1422.435000
CVaR 95% approx. : $1269.738221

andhi@22036-Nara MINGW64 /c/Users/andhi/Documents/GitHub/HPC_MonteCarlo
$

```

Fig. B6. MPI execution output using 16 ranks.